Filter Pandas Dataframe with multiple conditions

The reason is Dataframe may be having multiple columns and multiple rows. Selective display of columns with limited rows is always the expected view of users. To fulfill the user's expectations and also help in machine deep learning scenarios, filtering of Pandas Dataframe with multiple conditions is much necessary

```
import pandas as pd
# assign data
df = pd.DataFrame({'Name': ['RACHEL', 'MONICA',\
                            'PHOEBE','ROSS', 'CHANDLER','JOEY'],\
                   'Age':[30,35,37,33,34,30],\
                   'Salary':[100000,93000,88000,120000,94000,95000],\
                   'Job':['DESIGNER','CHEF','MASUS','PALENTOLOGY',\
                          'IT','ARTIST']})
# display dataframe
print(df)
```

Output:

	Name	Age	Salary	Job
0	RACHEL	30	100000	DESIGNER
1	MONICA	35	93000	CHEF
2	PHOEBE	37	88000	MASUS
3	ROSS	33	120000	PALENTOLOGY
4	CHANDLER	34	94000	IT
5	JOEY	30	95000	ARTIST

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

1. Filter Pandas Dataframe with multiple conditions Using loc

Here we will get all rows having Salary greater or equal to 100000 and Age < 40 and their JOB starts with 'D' from the dataframe. Print the details with Name and their JOB. For the above requirement, we can achieve this by using **loc**. It is used to access single or more rows and columns by label(s) or by a boolean array. loc works with column labels and indexes.

Filter Name and Job from data with salary greater than 100000 age less than 40 job starting with D with loc method

```
import pandas as pd
# assign data
df = pd.DataFrame({'Name': ['RACHEL', 'MONICA', 'PHOEBE', 'ROSS',\
                            'CHANDLER','JOEY'],\
                   'Age': [30,35,37,33,34,30],\
                   'Salary': [100000,93000,88000,120000,94000,95000],\
                   'Job': ['DESIGNER','CHEF','MASUS','PALENTOLOGY',\
                           'IT','ARTIST']})
print(df.loc[(df['Salary']>=100000)&
             (df['Age']<40)&\
             (df['Job'].str.startswith('D')),\
             ['Name','Job']])
```

Output:

```
      Name       Job
0   RACHEL   DESIGNER
```

smetechnologies
GUIDING THE FUTURE WITH AI
Ph. No: +91 9778191215

2. Filter Pandas Dataframe Using NumPy

Here will get all rows having Salary greater or equal to 100000 and Age < 40 and their JOB starts with 'D' from the data frame. We need to use NumPy.

Filter Name and Job from data with salary greater than 100000 age less than 40 job starting with D with numpy where method

```
import pandas as pd
import numpy as np

# assign data
df =pd.DataFrame({'Name':['RACHEL','MONICA','PHOEBE','ROSS',\
                          'CHANDLER','JOEY'],\
                  'Age': [30,35,37,33,34,30],\
                 'Salary': [100000,93000,88000,120000,94000,95000],\
                'Job': ['DESIGNER','CHEF','MASUS','PALENTOLOGY',\
                        'IT','ARTIST']})
filtered_values = np.where((df['Salary']>=100000)&\
                           (df['Age']<40)&\
                           (df['Job'].str.startswith('D')))
print(df.loc[filtered_values])
```

Output:

	Name	Age	Salary	Job
0	*RACHEL*	*30*	*100000*	*DESIGNER*

smetechnologies
GUIDING THE IT'S WITH
Ph. No: +91 9778102153

3. Filter Pandas Dataframe Using Query (eval and query works only with columns)

In this approach, we get all rows having Salary lesser or equal to 100000 and Age < 40, and their JOB starts with 'D' from the dataframe. Its just query the columns of a DataFrame with a single or more Boolean expressions and if multiple, it is having & condition in the middle.

Filter Name and Job from data with salary greater than 100000 age less than 40 job starting with D with query method

```
import pandas as pd
import numpy as np

# assign data
df=pd.DataFrame({'Name':['RACHEL','MONICA','PHOEBE','ROSS',\
                         'CHANDLER','JOEY'],\
                 'Age': [30,35,37,33,34,30],\
                 'Salary': [100000,93000,88000,120000,94000,95000],
                 'Job':['DESIGNER','CHEF','MASUS','PALENTOLOGY',\
                        'IT','ARTIST']})
print(df.query('Salary <=100000& \
                Age<40& \
                Job.str.startswith("D").values'))
```

Output:

```
    Name  Age  Salary      Job
0  RACHEL   30  100000  DESIGNER
```

smetechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

4. Pandas Boolean indexing multiple conditions standard way ("Boolean indexing" works with values in a column only)

In this approach, we get all rows having Salary lesser or equal to 100000 and Age < 40 and their JOB starts with 'D' from the dataframe. In order to select the subset of data using the values in the dataframe and applying Boolean conditions, we need to follow these ways

Filter Name and Job from data with salary greater than 100000 age less than 40 job starting with D with pandas filtering

```
import pandas as pd
import numpy as np

# assign data
df=pd.DataFrame({'Name':['RACHEL','MONICA','PHOEBE','ROSS',\
                         'CHANDLER','JOEY'],\
                 'Age': [30,35,37,33,34,30],\
                 'Salary': [100000,93000,88000,120000,94000,95000],
                 'Job': ['DESIGNER','CHEF','MASUS','PALENTOLOGY',\
                         'IT','ARTIST']})
print(df[(df['Salary']>=100000)&(df['Age']<40)&df['Job'].str.startswit
h('D')][['Name','Age','Salary']])
```

Output:

	Name	Age	Salary
0	RACHEL	30	100000

smctechnologies
II Floor, Empress House, Metro Junction, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9777091255
GUIDING THE FUTURE WITH AI

5. Eval multiple conditions ("eval" and "query" works only with columns)

Here, we get all rows having Salary lesser or equal to 100000 and Age < 40 and their JOB starts with 'D' from the dataframe.

Filter Name and Job from data with salary greater than 100000 age less than 40 job starting with D with eval method

```
import pandas as pd
import numpy as np

# assign data
df=pd.DataFrame({'Name':['RACHEL','MONICA','PHOEBE','ROSS',\
                         'CHANDLER','JOEY'],\
                 'Age': [30,35,37,33,34,30],\
                 'Salary': [100000,93000,88000,120000,94000,95000],\
                 'Job': ['DESIGNER','CHEF','MASUS','PALENTOLOGY',\
                         'IT','ARTIST']})
print(df[df.eval("Salary<=100000 & \
                 Age<40 & \
                 Job.str.startswith('D').values")])
```

Output:

```
     Name  Age  Salary       Job
0  RACHEL   30  100000  DESIGNER
```

smctechnologies
CRAFTING THE FUTURE WITH AI
II Floor, Express House, Palarivattom, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191225

eval method can also be used to perform arithmetic operations

```
import pandas as pd
import numpy as np

# assign data
df=pd.DataFrame({'Name':['RACHEL','MONICA','PHOEBE','ROSS',\
                         'CHANDLER','JOEY'],\
                 'Age': [30,35,37,33,34,30],\
                'Salary': [100000,93000,88000,120000,94000,95000],\
                'Job': ['DESIGNER','CHEF','MASUS','PALENTOLOGY',\
                        'IT','ARTIST']})
df['Salary_in_Thousands']=df.eval('Salary /1000')
print(df)
```

Output:

	Name	Age	Salary	Job	Salary_in_Thousands
0	RACHEL	30	100000	DESIGNER	100.0
1	MONICA	35	93000	CHEF	93.0
2	PHOEBE	37	88000	MASUS	88.0
3	ROSS	33	120000	PALENTOLOGY	120.0
4	CHANDLER	34	94000	IT	94.0
5	JOEY	30	95000	ARTIST	95.0

smectechnologies
CODING THE FUTURE WITH AI
II Floor, Espress House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778801215

6. How to Filter DataFrame by List of Values in Pandas

Here, we get filtered rows based on the list of values mentioned in isin().

Filter Name and Job from data using isin method

```
import pandas as pd
import numpy as np

# assign data
df=pd.DataFrame({'Name':['RACHEL','MONICA','PHOEBE','ROSS',\
                         'CHANDLER','JOEY'],\
                 'Age': [30,35,37,33,34,30],\
                 'Salary': [100000,93000,88000,120000,94000,95000],\
                 'Job': ['DESIGNER','CHEF','MASUS','PALENTOLOGY',\
                         'IT','ARTIST']})
job_list = ['DESIGNER', 'ARTIST']

# Filter DataFrame by list of job list
filtered_df = df[df['Job'].isin(job_list)]
print(filtered_df)
```

Output:

```
     Name   Age  Salary      Job
0   RACHEL   30  100000  DESIGNER
5    JOEY    30   95000    ARTIST
```

7. How to Filter DataFrame using filter in Pandas

Filter Name and Job from data using filter method

```
import pandas as pd
import numpy as np

# assign data
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 7819 915

```
df=pd.DataFrame({'Name':['RACHEL','MONICA','PHOEBE','ROSS',\
                         'CHANDLER','JOEY'],\
                 'Age': [30,35,37,33,34,30],\
                'Salary': [100000,93000,88000,120000,94000,95000],\
               'Job': ['DESIGNER','CHEF','MASUS','PALENTOLOGY',\
                       'IT','ARTIST']})
# Select specific columns using the 'items' parameter
filtered_df = df.filter(items=['Name', 'Age'])
print(filtered_df)
```

Output:

	Name	Age
0	RACHEL	30
1	MONICA	35
2	PHOEBE	37
3	ROSS	33
4	CHANDLER	34
5	JOEY	30

8.Selecting Columns Using filter and like

Filter columns containing ‘a’ from data using filter method and like

```
import pandas as pd
import numpy as np

# assign data
df=pd.DataFrame({'Name':['RACHEL','MONICA','PHOEBE','ROSS',\
                         'CHANDLER','JOEY'],\
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Phone: +91 9778191215

```
                    'Age': [30,35,37,33,34,30],\
                    'Salary': [100000,93000,88000,120000,94000,95000],\
                    'Job': ['DESIGNER','CHEF','MASUS','PALENTOLOGY',\
                           'IT','ARTIST']})
# Select columns whose names contain 'a'
filtered_df = df.filter(like='a', axis=1)
print(filtered_df)
```

Output:

	Name	Salary
0	RACHEL	100000
1	MONICA	93000
2	PHOEBE	88000
3	ROSS	120000
4	CHANDLER	94000
5	JOEY	95000

9. Selecting Columns Using regex

Filter columns starting with A from data using filter method and regex

```
import pandas as pd
import numpy as np

# assign data
df=pd.DataFrame({'Name':['RACHEL','MONICA','PHOEBE','ROSS',\
                         'CHANDLER','JOEY'],\
                    'Age': [30,35,37,33,34,30],\
                    'Salary': [100000,93000,88000,120000,94000,95000],\
```

smctechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 7907819215

```
                'Job': ['DESIGNER','CHEF','MASUS','PALENTOLOGY',\
                        'IT','ARTIST']})
filtered_df = df.filter(regex='^A', axis=1)
print(filtered_df)
```

Output:

```
   Age
0   30
1   35
2   37
3   33
4   34
5   30
```

Merging/Joining

Pandas provides a single function, merge, as the entry point for all standard database join operations between DataFrame objects

Syntax:

pd.merge(left, right, how='inner', on=None, left_on=None, right_on=None, left_index=False, right_index=False, sort=True)

Here, we have used the following parameters −

left − A DataFrame object.

right − Another DataFrame object.

on − Columns (names) to join on. Must be found in both the left and right DataFrame objects.

smetechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 7781912135

left_on − Columns from the left DataFrame to use as keys. Can either be column names or arrays with length equal to the length of the DataFrame.

right_on − Columns from the right DataFrame to use as keys. Can either be column names or arrays with length equal to the length of the DataFrame.

left_index − If True, use the index (row labels) from the left DataFrame as its join key(s). In case of a DataFrame with a MultiIndex (hierarchical), the number of levels must match the number of join keys from the right DataFrame.

right_index − Same usage as left_index for the right DataFrame.

how − One of 'left', 'right', 'outer', 'inner'. Defaults to inner. Each method has been described below.

sort − Sort the result DataFrame by the join keys in lexicographical order. Defaults to True, setting to False will improve the performance substantially in many cases.

Let us now create two different DataFrames and perform the merging operations on it.

```
# import the pandas library
import pandas as pd
left = pd.DataFrame({
   'id':[1,2,3,4,5],
   'Name': ['Alex', 'Amy', 'Allen', 'Alice', 'Ayoung'],
   'subject_id':['sub1','sub2','sub4','sub6','sub5']})
right = pd.DataFrame({'id':[1,2,3,4,5],\
    'Name': ['Billy', 'Brian', 'Bran', 'Bryce', 'Betty'],\
```

Ii Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala (+91 9778191215
Ph. No: +91 9778191215

```
    'subject_id':['sub2','sub4','sub3','sub6','sub5']})
print(left)
print(right)
```

Output:

```
  id     Name subject_id
0   1     Alex       sub1
1   2      Amy       sub2
2   3    Allen       sub4
3   4    Alice       sub6
4   5   Ayoung       sub5
   id    Name subject_id
0   1   Billy       sub2
1   2   Brian       sub4
2   3   Bran       sub3
3   4   Bryce       sub6
4   5   Betty       sub5
```

Merge Two DataFrames on a Key

```
import pandas as pd
left = pd.DataFrame({
    'id':[1,2,3,4,5],
    'Name': ['Alex', 'Amy', 'Allen', 'Alice', 'Ayoung'],
    'subject_id':['sub1','sub2','sub4','sub6','sub5']})
right = pd.DataFrame({'id':[1,2,3,4,5],\
    'Name': ['Billy', 'Brian', 'Bran', 'Bryce', 'Betty'],\
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
   'subject_id':['sub2','sub4','sub3','sub6','sub5']})
print(left)
print(right)
print(pd.merge(left,right,on='id'))
```

Output:

```
 id     Name subject_id
0   1     Alex       sub1
1   2      Amy       sub2
2   3    Allen       sub4
3   4    Alice       sub6
4   5   Ayoung       sub5
   id    Name subject_id
0   1   Billy       sub2
1   2   Brian       sub4
2   3    Bran       sub3
3   4   Bryce       sub6
4   5   Betty       sub5
   id  Name_x subject_id_x Name_y subject_id_y
0   1    Alex         sub1  Billy         sub2
1   2     Amy         sub2  Brian         sub4
2   3   Allen         sub4   Bran         sub3
3   4   Alice         sub6  Bryce         sub6
4   5  Ayoung         sub5  Betty         sub5
```

smetechnologies
GUIDING THE FUTURE WITH AI
1st Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph No: +91 9746191215

Merge Two DataFrames on Multiple Keys

```
import pandas as pd
left = pd.DataFrame({
    'id':[1,2,3,4,5],
    'Name': ['Alex', 'Amy', 'Allen', 'Alice', 'Ayoung'],
    'subject_id':['sub1','sub2','sub4','sub6','sub5']})
right = pd.DataFrame({'id':[1,2,3,4,5],\
    'Name': ['Billy', 'Brian', 'Bran', 'Bryce', 'Betty'],\
    'subject_id':['sub2','sub4','sub3','sub6','sub5']})
print(left)
print(right)
print(pd.merge(left,right,on=['id','subject_id']))
```

Output:

```
id    Name subject_id
0   1    Alex       sub1
1   2     Amy       sub2
2   3   Allen       sub4
3   4   Alice       sub6
4   5  Ayoung       sub5
   id   Name subject_id
0   1  Billy       sub2
1   2  Brian       sub4
2   3   Bran       sub3
3   4  Bryce       sub6
4   5  Betty       sub5
   id  Name_x subject_id Name_y
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
0   4   Alice       sub6  Bryce
1   5  Ayoung       sub5  Betty
```

Merge Using 'how' Argument

The how argument to merge specifies how to determine which keys are to be included in the resulting table. If a key combination does not appear in either the left or the right tables, the values in the joined table will be NA.

Merge Method	SQL Equivalent	Description
left	LEFT OUTER JOIN	Use keys from left object
right	RIGHT OUTER JOIN	Use keys from right object
outer	FULL OUTER JOIN	Use union of keys
inner	INNER JOIN	Use intersection of keys

Left Join

```
import pandas as pd
left = pd.DataFrame({'id':[1,2,3,4,5],\
   'Name': ['Alex', 'Amy', 'Allen', 'Alice', 'Ayoung'],\
   'subject_id':['sub1','sub2','sub4','sub6','sub5']})
right = pd.DataFrame({'id':[1,2,3,4,5],\
   'Name': ['Billy', 'Brian', 'Bran', 'Bryce', 'Betty'],\
   'subject_id':['sub2','sub4','sub3','sub6','sub5']})
print(left)
print(right)
print(pd.merge(left, right, on='subject_id', how='left'))
```

```
Output:
id    Name subject_id
0   1    Alex       sub1
1   2     Amy       sub2
2   3   Allen       sub4
3   4   Alice       sub6
4   5  Ayoung       sub5
   id   Name subject_id
0   1  Billy       sub2
1   2  Brian       sub4
2   3   Bran       sub3
3   4  Bryce       sub6
4   5  Betty       sub5
   id_x  Name_x subject_id  id_y Name_y
0     1    Alex       sub1   NaN    NaN
1     2     Amy       sub2   1.0  Billy
2     3   Allen       sub4   2.0  Brian
3     4   Alice       sub6   4.0  Bryce
4     5  Ayoung       sub5   5.0  Betty
```

Right Join

```
import pandas as pd
left = pd.DataFrame({'id':[1,2,3,4,5],\
   'Name': ['Alex', 'Amy', 'Allen', 'Alice', 'Ayoung'],\
   'subject_id':['sub1','sub2','sub4','sub6','sub5']})
right = pd.DataFrame({'id':[1,2,3,4,5],\
```

smartechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express Bussiness Kaloor Kochi, Ernakulam, Kerala 682017
Ph. No: +91 8078 215

```
    'Name': ['Billy', 'Brian', 'Bran', 'Bryce', 'Betty'],\
    'subject_id':['sub2','sub4','sub3','sub6','sub5']})
print(left)
print(right)
print(pd.merge(left, right, on='subject_id', how='right'))
```

Output:

```
id    Name subject_id
0   1    Alex       sub1
1   2     Amy       sub2
2   3   Allen       sub4
3   4   Alice       sub6
4   5  Ayoung       sub5
   id   Name subject_id
0   1  Billy       sub2
1   2  Brian       sub4
2   3   Bran       sub3
3   4  Bryce       sub6
4   5  Betty       sub5
   id_x  Name_x subject_id  id_y Name_y
0   2.0     Amy       sub2     1  Billy
1   3.0   Allen       sub4     2  Brian
2   NaN     NaN       sub3     3   Bran
3   4.0   Alice       sub6     4  Bryce
4   5.0  Ayoung       sub5     5  Betty
```

smectechnologies
GUIDING THE FUTURE WITH AI
U1 Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph No: +91 7907819215

Outer Join

```
import pandas as pd
left = pd.DataFrame({'id':[1,2,3,4,5],\
   'Name': ['Alex', 'Amy', 'Allen', 'Alice', 'Ayoung'],\
   'subject_id':['sub1','sub2','sub4','sub6','sub5']})
right = pd.DataFrame({'id':[1,2,3,4,5],\
   'Name': ['Billy', 'Brian', 'Bran', 'Bryce', 'Betty'],
   'subject_id':['sub2','sub4','sub3','sub6','sub5']})
print(left)
print(right)
print(pd.merge(left, right, how='outer', on='subject_id'))
```

Output:

```
id    Name subject_id
0   1    Alex       sub1
1   2     Amy       sub2
2   3   Allen       sub4
3   4   Alice       sub6
4   5  Ayoung       sub5
   id   Name subject_id
0   1  Billy       sub2
1   2  Brian       sub4
2   3   Bran       sub3
3   4  Bryce       sub6
4   5  Betty       sub5
   id_x  Name_x subject_id  id_y Name_y
```

smectechnologies
GUIDING THE FUTURE WITH IT
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala - 682017
Ph. No: +91 9778191215

```
0   1.0    Alex       sub1   NaN    NaN
1   2.0     Amy       sub2   1.0  Billy
2   3.0   Allen       sub4   2.0  Brian
3   4.0   Alice       sub6   4.0  Bryce
4   5.0  Ayoung       sub5   5.0  Betty
5   NaN     NaN       sub3   3.0   Bran
```

Inner Join

Joining will be performed on index. Join operation honors the object on which it is called. So, a.join(b) is not equal to b.join(a).

```
import pandas as pd
left = pd.DataFrame({'id':[1,2,3,4,5],\
   'Name': ['Alex', 'Amy', 'Allen', 'Alice', 'Ayoung'],\
   'subject_id':['sub1','sub2','sub4','sub6','sub5']})
right = pd.DataFrame({'id':[1,2,3,4,5],\
   'Name': ['Billy', 'Brian', 'Bran', 'Bryce', 'Betty'],\
   'subject_id':['sub2','sub4','sub3','sub6','sub5']})
print(left)
print(right)
print(pd.merge(left, right, on='subject_id', how='inner'))
```

```
Output:
id    Name subject_id
0   1    Alex       sub1
1   2     Amy       sub2
2   3   Allen       sub4
```

smetechnologies
GUIDING THE FUTURE WITH AI
4th Floor, Express House, Kaloor Kochi, Ernakulam, Kerala 682017
Ph No: +91 9778192215

```
3   4   Alice       sub6
4   5  Ayoung       sub5
   id   Name subject_id
0   1  Billy       sub2
1   2  Brian       sub4
2   3   Bran       sub3
3   4  Bryce       sub6
4   5  Betty       sub5
   id_x  Name_x subject_id  id_y Name_y
0     2     Amy       sub2     1  Billy
1     3   Allen       sub4     2  Brian
2     4   Alice       sub6     4  Bryce
3     5  Ayoung       sub5     5  Betty
```

Dataframe Concatenation

Pandas provides various facilities for easily combining together Series and DataFrame

Syntax:

```
pd.concat(objs,axis=0,join='outer',join_axes=None,
ignore_index=False)
```

objs − This is a sequence or mapping of Series or DataFrame. axis − {0, 1, ...}, default 0. This is the axis to concatenate along.

join − {‘inner’, ‘outer’}, default ‘outer’. How to handle indexes on other axis(es). Outer for union and inner for intersection.

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 7907819215

ignore_index − boolean, default False. If True, do not use the index values on the concatenation axis. The resulting axis will be labeled 0, ..., n - 1.

join_axes − This is the list of Index objects. Specific indexes to use for the other (n-1) axes instead of performing inner/outer set logic.

Concatenating Objects

The concat function does all of the heavy lifting of performing concatenation operations along an axis

```
import pandas as pd
one = pd.DataFrame({\
   'Name': ['Alex', 'Amy', 'Allen', 'Alice', 'Ayoung'],\
   'subject_id':['sub1','sub2','sub4','sub6','sub5'],\
   'Marks_scored':[98,90,87,69,78]},\
   index=[1,2,3,4,5])

two = pd.DataFrame({\
   'Name': ['Billy', 'Brian', 'Bran', 'Bryce', 'Betty'],\
   'subject_id':['sub2','sub4','sub3','sub6','sub5'],\
   'Marks_scored':[89,80,79,97,88]},\
   index=[1,2,3,4,5])
print(pd.concat([one,two]))
```

Output:

```
Name subject_id  Marks_scored
```

Insnectechnologies
GUIDING THE FUTURE WITH AI
1st Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph No: +91 9778191215

```
1    Alex       sub1            98
2     Amy       sub2            90
3   Allen       sub4            87
4   Alice       sub6            69
5  Ayoung       sub5            78
1   Billy       sub2            89
2   Brian       sub4            80
3    Bran       sub3            79
4   Bryce       sub6            97
5   Betty       sub5            88
```

If two objects need to be added along axis=1, then the new columns will be appended.

```
import pandas as pd
one = pd.DataFrame({\
   'Name': ['Alex', 'Amy', 'Allen', 'Alice', 'Ayoung'],\
   'subject_id':['sub1','sub2','sub4','sub6','sub5'],\
   'Marks_scored':[98,90,87,69,78]},\
   index=[1,2,3,4,5])

two = pd.DataFrame({\
   'Name': ['Billy', 'Brian', 'Bran', 'Bryce', 'Betty'],\
   'subject_id':['sub2','sub4','sub3','sub6','sub5'],\
   'Marks_scored':[89,80,79,97,88]},\
   index=[1,2,3,4,5])
```

smetechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 97781 92153

```
print(pd.concat([one,two],axis=1))
```

Output:

	Name	subject_id	Marks_scored	Name	subject_id	Marks_scored
1	Alex	sub1	98	Billy	sub2	89
2	Amy	sub2	90	Brian	sub4	80
3	Allen	sub4	87	Bran	sub3	79
4	Alice	sub6	69	Bryce	sub6	97
5	Ayoung	sub5	78	Betty	sub5	88

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215